

LABORATORIO PYTHON + ANALISI DATI

Dall'Elaborazione Dati con Pandas alle Dashboard Interattive in Produzione

■ **Ente:** ITIS Campobasso | ■■ **Durata:** 22 Ore (80% Laboratorio Hands-on / 20% Teoria)

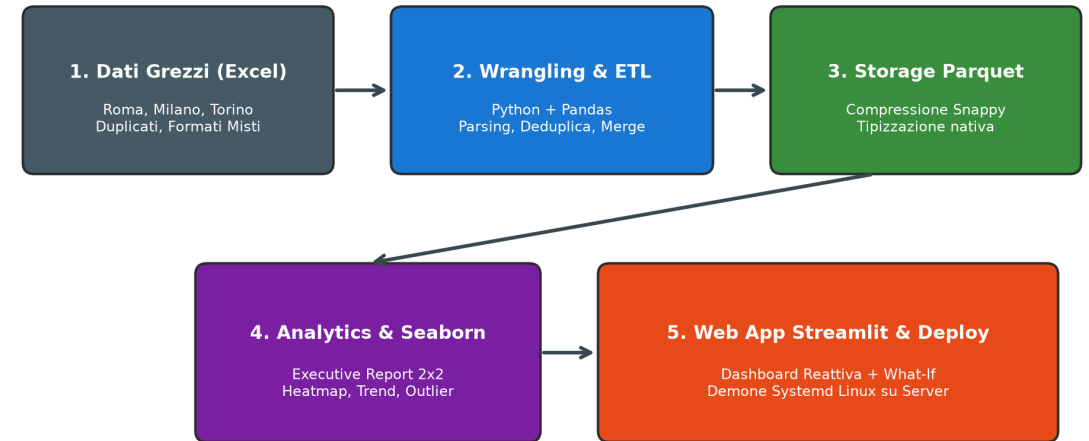
■■■ **Docente:** Arnaldo Morena — Senior Python Developer & Data Architect

■ **Repository:** https://github.com/arnymore/python_analisi_dati_campobasso

Il Caso Aziendale Unico: Rete Commerciale Italia

■ Scenario di Business

- Azienda commerciale nazionale con 4 sedi: Roma, Milano, Torino e Napoli.
- Catalogo prodotti articolato: Hardware, Software, Servizi IT e Cancelleria.
- I gestionali regionali estraggono file Excel non standardizzati e con anomalie.
- Obiettivo: Costruire un sistema automatizzato end-to-end per consolidare e analizzare i KPI.



Tipi di Dato Primitivi e Casting nei Dati Aziendali

■ Tipizzazione Dinamica e Pulizia

- Stringhe (str): Codici cliente, nomi prodotti, date testuali.
- Interi (int) e Float (float): Pezzi venduti, importi monetari e sconti.
- Booleani (bool): Flag transazione approvata / pagamento saldato.
- Attenzione al 'Dirty Casting': stringhe con virgola '1250,50 €' falliscono con float() se non ripulite preventivamente.

■ Scripting: Sanitizzazione Stringhe Prezzo

```
# Esempio di pulizia e cast monetario
prezzo_grezzo = ' 1.850,50 € '
prezzo_clean = prezzo_grezzo.replace('€', '') \
                        .replace('.', '') \
                        .replace(',', '.') \
                        .strip()

prezzo_num = float(prezzo_clean)
print(f'Prezzo numerico: {prezzo_num:.2f}')
# Output: Prezzo numerico: 1850.50
```

Strutture Dati Native: Liste, Tuple e Dizionari

■ ■ Modellare Record in Memoria

- Liste []: Sequenze ordinate e modificabili (es. carrello ordini).
- Tuple (): Record a sola lettura immutabili (es. coordinate geografiche).
- Dizionari {}: Mappature chiave-valore per rappresentare righe di tabelle.
- Accesso sicuro con `.get('chiave', default)` per evitare `KeyError` su campi opzionali.

■ Dizionario per Rappresentare Transazioni

```
# Modellazione di una riga transazione
ordine = {
    'id': 'R0-2024001',
    'cliente': 'Tech Solutions Srl',
    'prodotto': 'Server Rack 24U',
    'qta': 2,
    'prezzo': 1850.00,
    'sconto': 10
}
print(ordine.get('note', 'Nessuna nota'))
```

Logica Condizionale, Cicli ed Iterazioni

■ ■ Logica Commerciale nei Cicli

- Costrutti if / elif / else per applicare scaglioni di sconto e rating cliente.
- Ciclo for per iterare su elenchi di vendite ed accumulare totali progressivi.
- Funzioni enumerate() per contatori e zip() per accoppiare liste parallele.
- List Comprehension: trasformazioni ed estrazioni sintetiche ad alte prestazioni.

■ Elaborazione Multi-Riga con Comprehension

```
ordini = [  
    {'prod': 'Server', 'qta': 2, 'prezzo': 1850},  
    {'prod': 'Mouse', 'qta': 5, 'prezzo': 25},  
    {'prod': 'Laptop', 'qta': 1, 'prezzo': 900}  
]  
  
# Filtro compatto con List Comprehension  
high_value = [o for o in ordini if o['prezzo'] > 500]  
totale = sum(o['qta'] * o['prezzo'] for o in ordini)  
print(f'Fatturato Lordo: {totale:.2f} €')
```

Modularità: Funzioni per il Calcolo di KPI Commerciali

■ Architettura a Funzioni

- Incapsulamento della logica aziendale: mai duplicare formule contabili.
- Parametri posizionali e valori di default (es. aliquota_iva=0.22, sconto_perc=0).
- Ritorno strutturato: restituire dizionari con tutti i dettagli riga calcolati.
- Docstring di documentazione per rendere il codice auto-esplicativo in team.

■ Funzione di Calcolo Riga Fattura

```
def calcola_totale_riga(qta, prezzo, sconto=0, iva=0.22):  
    """Calcola imponibile, sconto, IVA e totale."""  
    lordo = qta * prezzo  
    val_sconto = lordo * (sconto / 100.0)  
    netto = lordo - val_sconto  
    imposta = netto * iva  
    return {  
        'netto': round(netto, 2),  
        'iva': round(imposta, 2),  
        'totale': round(netto + imposta, 2)  
    }
```

Laboratorio 1: Calcolo KPI e Normalizzazione Anagrafiche

■ Consegne per gli Studenti

- File di lavoro: laboratori/lab01_python_operativo/lab01_esercizi.py
- Task 1: Implementare `calcola_totale_riga()` con arrotondamenti a 2 decimali.
- Task 2: Calcolare i totali netti e IVA su un carrello ordini multi-prodotto.
- Task 3: Scrivere `normalizza_ragione_sociale()` per ripulire nomi con spazi e maiuscole.
- Task 4: Raggruppare le vendite per categoria con dizionario accumulatore.

■ Output di Verifica (lab01_esercizi.py)

```
--- OUTPUT ATTESO DALL'ESECUZIONE ---  
Risultato riga: {'netto': 3825.0, 'iva': 841.5, ...}  
Fatturato Netto Ordine: 7522.50 €  
Totale IVA Ordine:      1654.95 €  
Clienti normalizzati:  
  • ' tech solutions srl ' -> 'Tech Solutions Srl'  
  • '  logistica  molise  spa ' -> 'Logistica Molise SpA'  
Hardware: Qta 11 | Fatturato: 9070.00 €  
Software: Qta 13 | Fatturato: 5100.00 €
```

Recap Modulo 1: Competenze Acquisite & Prossimi Passi

■ Competenze Acquisite

- Piena padronanza di liste, dizionari e strutture annidate.
- Capacità di scrivere funzioni modulari per il calcolo contabile.
- Pulizia e normalizzazione delle stringhe anagrafiche con metodi nativi.
- Comprensione dei limiti delle strutture native su grandi volumi di dati.

■ Verso Pandas (Modulo 2)

- I cicli for nativi diventano lenti e verbosi oltre 10.000 righe.
- Nel Modulo 2 scopriremo i DataFrame di Pandas per manipolare milioni di righe istantaneamente.
- Tutte le operazioni fatte a mano verranno vettorializzate in una sola riga di codice.
- Prepararsi ad aprire il dataset Excel ufficiale della filiale di Roma!

Dal Foglio Excel al DataFrame Pandas

■ Perché Pandas per l'Analisi Dati?

- Excel limita a 1M righe e si blocca sui ricalcoli pesanti.
- Pandas offre strutture in memoria bidimensionali (DataFrame) e 1D (Series).
- Performance C/NumPy: calcoli vettorializzati 100x più veloci.
- Caricamento diretto con `pd.read_excel('dataset/raw/roma.xlsx')`.

■ Caricamento Dataset Excel

```
import pandas as pd
df_roma = pd.read_excel('dataset/raw/roma.xlsx', sheet_name='Dati_Vendite')
print(type(df_roma)) # DataFrame
print(df_roma.shape) # (1248 righe, 13 colonne)
```

Ispezione e Diagnostica Strutturale del DataFrame

■ Metodi di Ispezione Rapida

- `df.info()`: Visualizza colonne, conteggio non-nulli e tipi di dato.
- `df.describe()`: Statistiche descrittive (media, std, min, quartili, max).
- `df.head(n)` e `df.tail(n)`: Campiona le prime o ultime n righe.
- `df.columns` e `df.dtypes`: Ispezione nomi e formati memoria.

■ Ispezione Strutturale e Metadati

```
df_roma.info()  
print(df_roma[['Quantita', 'Sconto_Perc']].describe())  
print(df_roma.head(3))
```

Selezione Colonne: Series (1D) vs DataFrame (2D)

■ Estrazione Viste Dati

- Singola colonna `df['Cliente']` → Restituisce una Series 1D.
- Doppie quadre `df[['Cliente', 'Fatturato']]` → Restituisce un DataFrame 2D.
- Creazione di viste mirate per focalizzare l'analisi.
- Assegnazione a nuove variabili senza duplicare memoria.

■ Selezione Singola e Multi-Colonna

```
serie_clienti = df_roma['Ragione_Sociale']  
cols = ['ID_Transazione', 'Ragione_Sociale', 'Fatturato_Lordo']  
df_focus = df_roma[cols]  
print(df_focus.head(2))
```

Indicizzazione Avanzata: .iloc[] Posizionale vs .loc[] per Etichette

■ Accesso Diretto ai Dati

- `.iloc[righe, colonne]`: Indicizzazione numerica posizionale.
- `.loc[righe, colonne]`: Indicizzazione per etichetta o maschera booleana.
- Estremi inclusi in `.loc` vs estremo destro escluso in `.iloc`.
- Best practice: evitare il chaining `df['a']['b']` e usare sempre `.loc/.iloc`.

■ Slicing con .iloc e .loc

```
sub_pos = df_roma.iloc[10:16, 0:4]
sub_lbl = df_roma.loc[0:5, ['Ragione_Sociale', 'Fatturato_Lordo']]
print(sub_lbl)
```

Filtri Booleani e Maschere Logiche

■ Interrogare il Dataset

- Maschera booleana: `df['Canale_Vendita'] == 'E-commerce B2B'`.
- Restituisce un vettore di True e False.
- `df[maschera]` estrae solo le righe corrispondenti a True.
- Metodo `.sum()` sulla maschera per contare i record rispondenti.

■ Filtro su Singola Condizione

```
mask_ecom = df_roma['Canale_Vendita'] == 'E-commerce B2B'  
print(f'Ordini E-commerce: {mask_ecom.sum()}')  
df_ecom = df_roma[mask_ecom]  
print(df_ecom.head(2))
```

Condizioni Logiche Multiple: AND (&), OR (|), NOT (~)

■ Regole di Sintassi Vettoriale

- Usare sempre simboli bitwise &, |, ~ (non and/or/not).
- Ogni singola condizione DEVE essere tra parentesi: (cond1) & (cond2).
- AND (&): Entrambe vere; OR (|): Almeno una vera; NOT (~): Inversione.
- Filtro congiunto su volumi e sconti applicati.

■ Filtri Booleani Composti

```
cond_qta = (df_roma['Quantita'] >= 10)
cond_sco = (df_roma['Sconto_Perc'] > 0)
df_scontati = df_roma[cond_qta & cond_sco]
print(f'Record filtrati: {len(df_scontati)}')
```

Filtri Avanzati: .isin() e Ricerca Testuale con .str.contains()

■ Pattern Matching su Dati

- `.isin([valori])`: Filtra record appartenenti a una lista.
- `.str.contains('keyword', case=False, na=False)`: Ricerca parziale.
- `na=False` previene errori su celle vuote (NaN).
- Indispensabile per estrarre famiglie di prodotti o canali.

■ Ricerche Testuali e Liste

```
canali = ['E-commerce B2B', 'Partner Commerciale']  
df_can = df_roma[df_roma['Canale_Vendita'].isin(canali)]  
df_srv = df_roma[df_roma['Nome_Prodotto'].str.contains('Server', case=False, na=False)]
```

Vettorializzazione e Colonne Calcolate

■ Calcoli Aritmetici Istantanei

- Nessun ciclo for riga per riga: operazioni vettoriali dirette.
- `df['Lordo'] = df['Quantita'] * df['Prezzo']` calcolato su tutto il DataFrame.
- Esecuzione compilata in C sottostante ad altissime prestazioni.
- Creazione di metriche aziendali derivate in una riga.

■ Calcolo Vettorializzato

```
df_calc = df_roma.copy()
df_calc['Lordo-Calcolato'] = df_calc['Quantita'] * 850.0
print(df_calc[['Quantita', 'Lordo-Calcolato']].head(3))
```

Cast Numerico Sicuro con `pd.to_numeric(errors='coerce')`

■ Gestione Errori di Conversione

- I prezzi con virgole o simboli '€' provocano crash con `float()`.
- `pd.to_numeric(..., errors='coerce')` trasforma i refusi in NaN senza bloccare l'esecuzione.
- Pulizia preliminare con `.str.replace('€', '').str.replace(',', '.')`.
- Diagnostica sicura delle celle anomale.

■ Conversione con Coerce a NaN

```
p_clean = df_roma['Prezzo_Unitario'].astype(str) \
    .str.replace('€', '', regex=False) \
    .str.replace(',', '.', regex=False).str.strip()
df_roma['Prezzo_Num'] = pd.to_numeric(p_clean, errors='coerce')
```

Calcolo Sconti, Margini e Fatturato Netto Finale

■ Definizione KPI Economici

- Totale Lordo = $\text{Quantita} * \text{Prezzo_Unitario_Num}$.
- Valore Sconto = $\text{Lordo} * (\text{Sconto_Perc} / 100)$.
- Fatturato Netto = $\text{Lordo} - \text{Valore Sconto}$.
- Arrotondamento coerente a 2 decimali.

■ Pipeline di Calcolo Finanziario

```
df_roma['Totale_Lordo'] = df_roma['Quantita'] * df_roma['Prezzo_Num']  
df_roma['Valore_Sconto'] = df_roma['Totale_Lordo'] * (df_roma['Sconto_Perc'] / 100.0)  
df_roma['Fatturato_Netto'] = round(df_roma['Totale_Lordo'] - df_roma['Valore_Sconto'], 2)
```

Ordinamento Dati con .sort_values() & Top Deals

■ Ranking e Analisi di Pareto

- `df.sort_values(by='Fatturato_Netto', ascending=False)` ordina decrescente.
- Ordinamento multi-chiave per Filiale e Netto.
- Estrazione dei Top 10 Contratti aziendali con `.head(10)`.
- Identificazione immediata dei clienti ad alto valore.

■ Estrazione Top 10 Contratti

```
top_10 = df_roma.sort_values(by='Fatturato_Netto', ascending=False) \
    [['ID_Transazione', 'Ragione_Sociale', 'Fatturato_Netto']].head(10)
print(top_10.to_string(index=False))
```

Statistiche Descrittive e Metriche di Posizione

■ Indicatori Statistici di Sintesi

- Media vs Mediana: la mediana è robusta contro outlier.
- `.sum()`: Fatturato totale complessivo.
- `.mean()` e `.median()`: Ticket medio e mediano.
- `.quantile([0.25, 0.50, 0.75])`: Analisi della dispersione.

■ Calcolo Metriche di Sintesi

```
tot = df_roma['Fatturato_Netto'].sum()
avg = df_roma['Fatturato_Netto'].mean()
med = df_roma['Fatturato_Netto'].median()
print(f'Totale: € {tot:,.2f} | Media: € {avg:,.2f} | Mediana: € {med:,.2f}')
```

Analisi delle Frequenze con .value_counts()

■ Distribuzione dei Volumi

- `df['Canale'].value_counts()` conta gli ordini per canale.
- Parametro `normalize=True` calcola le quote percentuali.
- `dropna=False` verifica l'integrità dei dati.
- Comprensione della composizione del mix distributivo.

■ Quote Canali di Vendita

```
quote = df_roma['Canale_Vendita'].value_counts(normalize=True) * 100
for c, pct in quote.items():
    print(f'{c:20}: {pct:5.1f}%')
```

Recap Modulo 2: Padronanza del DataFrame Pandas

■ Competenze Acquisite

- Caricamento ed esplorazione di file Excel complessi.
- Filtri booleani avanzati e manipolazione DataFrame.
- Vettorializzazione di calcoli finanziari e casting sicuro.
- Ordinamento, statistiche descrittive e ranking.

■ ■ Verso il Data Wrangling (Modulo 3)

Nel Modulo 3 affronteremo la bonifica del dato sporco.

Deduplicheremo record e risolveremo date miste e seriali Excel.

Imputeremo i valori nulli ed effettueremo le JOIN relazionali (`pd.merge`).

Anatomia del 'Dirty Data' nei Gestionali Aziendali

■ ■ I 4 Grandi Problemi dei Dati Reali

- 1. Record Duplicati: Transazioni registrate più volte.
- 2. Formati Data Misti: ISO, date IT, seriali Excel, mesi testuali.
- 3. Valori Mancanti (NaN): Prezzi o quantità omesse.
- 4. Categorie Sporche: 'HW', 'Softwre', 'Servizi IT'.

■ Strategia di Wrangling

```
# I 4 step della bonifica certificata:  
# 1. Deduplicazione con drop_duplicates()  
# 2. Normalizzazione testi e categorie  
# 3. Parsing date ibride e seriali Excel  
# 4. Imputazione condizionale con transform()
```

Rilevamento ed Eliminazione Record Duplicati

■ Pulizia dei Duplicati

- `df.duplicated()`: Identifica righe duplicate.
- `subset=['ID_Transazione']`: Controllo duplicati su chiave primaria.
- `df.drop_duplicates(keep='first')`: Mantiene la prima occorrenza.
- Verifica righe prima e dopo: tracciabilità record scartati.

■ Deduplicazione con `drop_duplicates`

```
n_init = len(df_roma)
df_clean = df_roma.drop_duplicates().copy()
n_rimossi = n_init - len(df_clean)
print(f'Rimossi {n_rimossi} duplicati. Righe pulite: {len(df_clean)}')
```

Standardizzazione Stringhe e Categorie Merceologiche

■ Pulizia Testuale e Normalizzazione

- Rimozione spazi con `.str.strip()` e maiuscole con `.str.upper()`.
- Mappatura delle categorie disallineate con funzioni `.apply()`.
- Riconduzione a 4 categorie standard (Hardware, Software, Servizi, Cancelleria).
- Eliminazione di categorie fittizie generate da refusi.

■ Funzione di Trascodifica Categorie

```
def normalizza_cat(val):  
    if pd.isna(val): return 'Altro'  
    s = str(val).strip().lower()  
    if 'hard' in s or 'hw' in s: return 'Hardware'  
    if 'soft' in s or 'sw' in s: return 'Software'  
    if 'serv' in s: return 'Servizi'  
    return 'Cancelleria' if 'canc' in s else 'Altro'  
df_clean['Categoria'] = df_clean['Categoria_Prodotto'].apply(normalizza_cat)
```

La Sfida delle Date: Gestione Formati Misti ed Excel Serials

■ Disomogeneità Temporale

- Excel memorizza le date come numero di giorni dal 30/12/1899.
- Nei file coesistono date ISO, formati IT ('15/03/2024') e seriali ('45506').
- Funzione ibrida con `datetime.date(1899, 12, 30) + timedelta(days=s)`.
- Conversione sicura al 100% in oggetti `datetime64`.

■ Algoritmo di Parsing Date Miste

```
import datetime
def parse_data_ibrida(val):
    if pd.isna(val): return pd.NaT
    s = str(val).strip()
    if s.isdigit(): # Seriale Excel
        dt = datetime.date(1899, 12, 30) + datetime.timedelta(days=int(s))
        return pd.to_datetime(dt)
    return pd.to_datetime(s, format='mixed', dayfirst=True)
```

Feature Engineering Temporale: Mesi, Trimestri e Giorni

■ Arricchimento del Dato Temporale

- Accessore `.dt` per estrarre componenti temporali.
- `.dt.year`, `.dt.month`, `.dt.day`: Componenti numeriche.
- `.dt.strftime('%b')`: Nomi mesi per grafici.
- `.dt.to_period('Q')`: Trimestre fiscale (2024Q1, 2024Q2).

■ Estrazione Componenti Temporali

```
df_clean['Data_dt'] = pd.to_datetime(df_clean['Data_Vendita'].apply(parse_data_ibrida))
df_clean['Anno'] = df_clean['Data_dt'].dt.year
df_clean['Mese'] = df_clean['Data_dt'].dt.month
df_clean['Trimestre'] = df_clean['Data_dt'].dt.to_period('Q').astype(str)
```

Trattamento Professionale dei Valori Mancanti (Imputazione)

■ Drop vs Imputazione Intelligente

- Cancellare righe riduce il campione e falsa i totali contabili.
- La media globale mescola prezzi di articoli diversi.
- Soluzione ottimale: Imputazione condizionale per gruppo con `transform('mean')`.
- Il prezzo mancante di un Laptop viene sostituito con la media dei Laptop.

■ Imputazione Condizionale con `transform()`

```
media_prod = df_clean.groupby('Nome_Prodotto')['Prezzo_Num'].transform('mean')
df_clean['Prezzo_Fin'] = df_clean['Prezzo_Num'].fillna(media_prod)
med_qta = df_clean.groupby('Nome_Prodotto')['Quantita'].transform('median')
df_clean['Quantita_Fin'] = df_clean['Quantita'].fillna(med_qta).astype(int)
```

Integrazione Dati: Concetti di JOIN Relazionale

■ Modello Relazionale dei Dati

- Tabella dei Fatti: Vendite (molte righe, chiave esterna Codice_Cliente).
- Tabella Dimensionale: Anagrafica Clienti (chiave primaria univoca).
- Left Join: Mantiene tutte le vendite e aggancia Settore, Sede e Rating.
- Controllo di cardinalità per evitare duplicazioni cartesiane.

■ Sintassi di Merge Relazionale

```
# Caricamento anagrafica clienti
df_anag = pd.read_excel('dataset/raw/roma.xlsx', sheet_name='Anagrafica_Clienti')
df_merged = pd.merge(df_clean, df_anag, on='Codice_Cliente', how='left')
print(f'Righe dopo merge: {len(df_merged)} (invariate)')
```

Implementazione della JOIN con pd.merge()

■ Parametri Chiave di pd.merge()

- `pd.merge(left, right, on='Codice_Cliente', how='left')`.
- Trattamento non censiti: `.fillna('Non Specificato')`.
- Arricchimento strategico del dataset con Rating creditizio e Settore.
- Base di partenza per le aggregazioni di business.

■ Esecuzione Left Join

```
df_merged = pd.merge(
    df_clean,
    df_anag[['Codice_Cliente', 'Settore', 'Citta_Sede', 'Rating_Affidabilita']],
    on='Codice_Cliente', how='left'
)
df_merged['Settore'] = df_merged['Settore'].fillna('Non Specificato')
```

Raggruppamenti Strategici con `groupby()` e `.agg()`

■ Il Paradigma Split-Apply-Combine

- Split: Raggruppamento per Settore o Canale.
- Apply: Calcolo somme, medie, conteggi.
- Combine: Ricomposizione tabella KPI.
- Named Aggregation con `.agg()`: ridenominazione colonne direttamente nel calcolo.

■ Named Aggregations con `.agg()`

```
rep_settore = df_merged.groupby('Settore').agg(  
    Fatturato_Totale=('Fatturato_Netto', 'sum'),  
    Numero_Ordini=('ID_Transazione', 'count'),  
    Sconto_Medio=('Sconto_Perc', 'mean')  
) .reset_index().sort_values('Fatturato_Totale', ascending=False)  
print(rep_settore.head(3).to_string(index=False))
```

Laboratorio 3: Data Cleaning Completo e Report Settori

■ Esercitazione Pratica Studenti

- File: laboratori/lab03_data_wrangling/lab03_esercizi.py
- Task 1: Eliminare 48 duplicati esatti.
- Task 2: Normalizzare le categorie merceologiche.
- Task 3: Effettuare il parsing del 100% delle date eterogenee.
- Task 4: Imputare prezzi e quantità condizionalmente.
- Task 5: Effettuare il merge con l'anagrafica clienti.

■ Output Atteso (lab03_esercizi.py)

Righe iniziali: 1248 | Duplicati: 48 | Rimanenti: 1200

Date valide: 1200 / 1200 (100% corrette)

Top Settori per Fatturato Netto:

1. Editoria & Cultura: € 468.706,11 (110 ordini)
2. Trasporti & Logistica: € 465.959,68 (102 ordini)
3. Information Technology: € 397.243,77 (95 ordini)

Tabelle Pivot Bidimensionali con `pd.pivot_table()`

■ Matrici di Dati Incrociate

- `pd.pivot_table(df, index='Righe', columns='Colonne', values='Valori', aggfunc='sum')`.
- Analisi incrociata Canale x Categoria.
- `fill_value=0` per sostituire vuoti con zeri.
- Base diretta per generare la Heatmap nel Modulo 4.

■ Costruzione Matrice Pivot

```
pivot_mat = pd.pivot_table(
    df_merged,
    index='Canale_Vendita', columns='Categoria',
    values='Fatturato_Netto', aggfunc='sum', fill_value=0
)
print((pivot_mat / 1000).round(1)) # in k€
```

Recap Modulo 3: Pipeline di Bonifica Dati Certificata

■ Competenze Acquisite

- Deduplicazione sicura e normalizzazione testi.
- Parsing robusto date miste e seriali Excel.
- Imputazione condizionale intelligente per prodotto.
- Join relazionale con anagrafiche e report Named Aggregations.

■ ■ Verso la Visualizzazione (Modulo 4)

I dati sono puliti, coerenti e certificati.

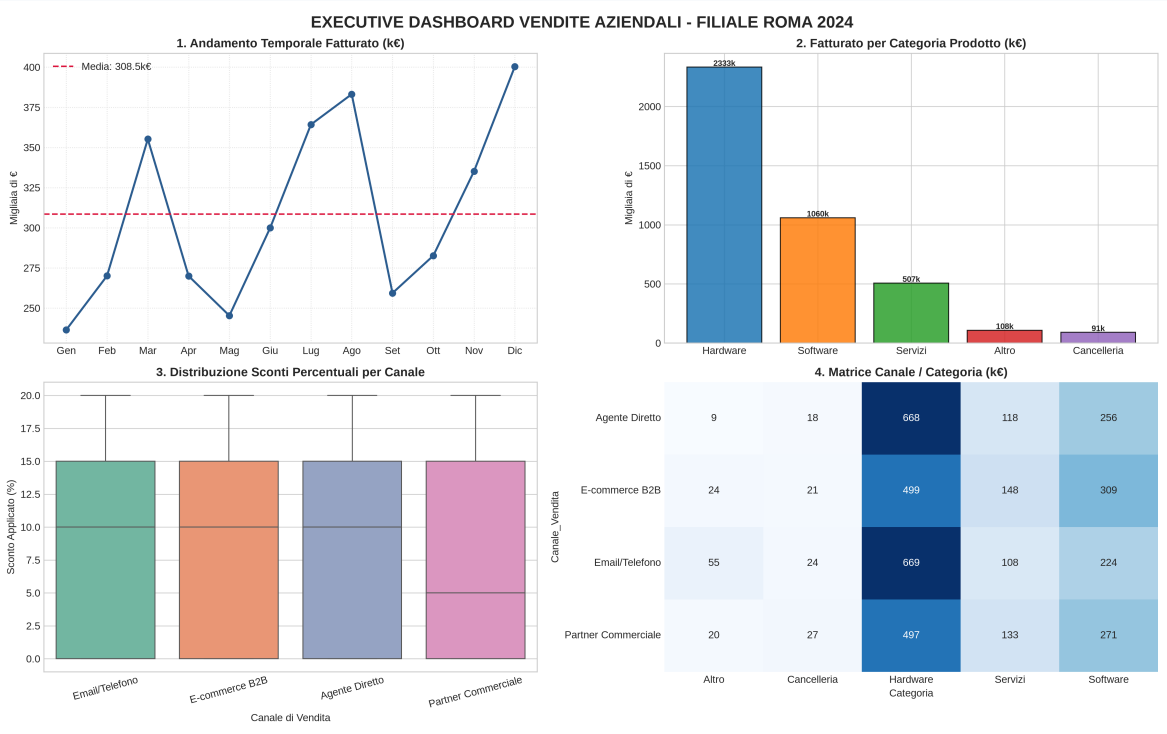
Nel Modulo 4 utilizzeremo Matplotlib e Seaborn.

Creeremo grafici a barre, trend temporali, heatmap ed Executive Report 2x2.

Principi di Data Storytelling e Visualizzazione Direzionale

■ Regole per Grafici Efficaci

- Eliminare il disordine visivo (Chartjunk): no 3D, no grafici fuorvianti.
- Scegliere il tipo di grafico in base all'obiettivo di business:
- • Confronti → Bar Chart orizzontale o verticale.
- • Serie storiche → Line Chart con marker.
- • Matrici incrociate → Heatmap con annotazioni.
- • Dispersione → Boxplot.
- Unità di misura sempre esplicitate (k€, %, M€).



Anatomia di Matplotlib: Figure e Axes (Approccio OOP)

■ Architettura Orientata a Oggetti

- Figure: Canvas contenitore generale.
- Axes: Singolo grafico con coordinate X/Y, titoli, griglia e tick.
- Sintassi: `fig, ax = plt.subplots(figsize=(9, 4.5))`.
- Controllo totale su layout, spaziature e DPI di esportazione.

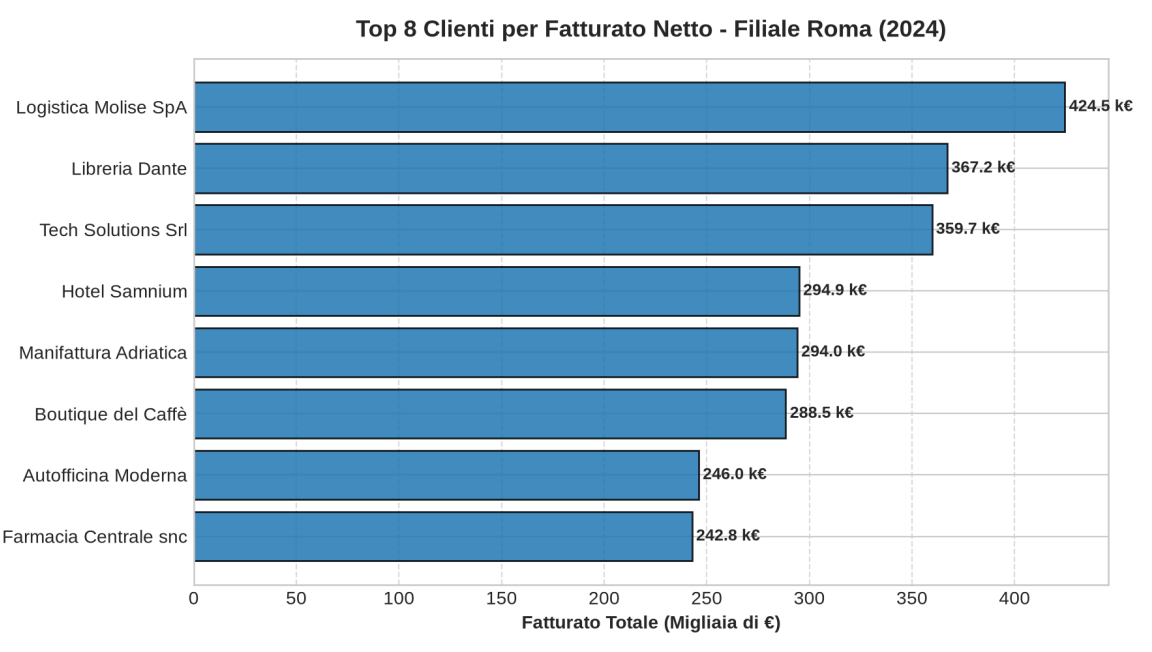
■ Sintassi ed Esempio Operativo

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 4), dpi=150)
ax.set_title('Titolo Grafico', fontweight='bold')
ax.set_xlabel('Asse X') | ax.set_ylabel('Asse Y')
ax.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
```

Bar Chart Orizzontale con Etichette Dati (Top Clienti)

■ Evidenziare i Key Accounts

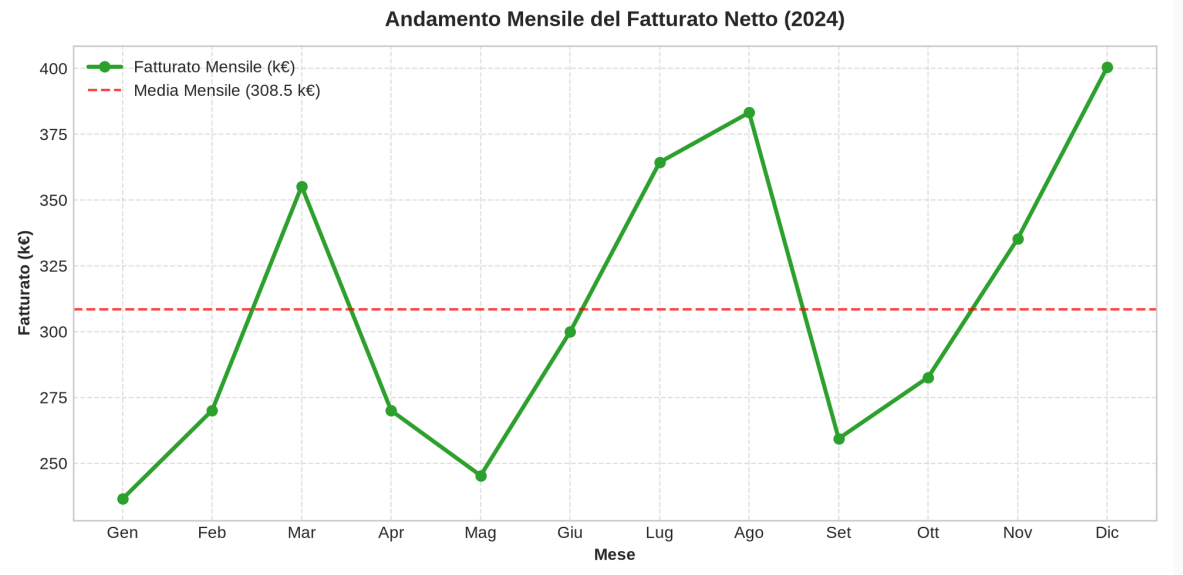
- ax.barh(clienti, valori) per etichette testuali lunghe.
- Valori espressi in k€ per facilitare la lettura.
- Etichette numeriche applicate direttamente su ogni barra.
- Ordinamento crescente per avere il top client in cima.



Serie Temporal: Trend Mensile con Linea di Media

■ Monitorare l'Andamento Temporale

- `ax.plot(mesi, valori, marker='o', lw=2.5)` per la curva di vendita.
- `ax.axhline(media, color='red', linestyle='--')` per il target annuale.
- Etichette dei mesi chiare (Gen..Dic) con `ax.set_xticklabels()`.
- Evidenziazione dei picchi e delle stagionalità commerciali.



Seaborn: Visualizzazioni Statistiche e Design Moderno

■ Eleganza e Statistica Avanzata

- Libreria di alto livello integrata con DataFrame Pandas.
- Temi preconfigurati: `sns.set_theme(style='whitegrid')`.
- Palette cromatiche aziendali: 'Blues', 'Set2', 'viridis'.
- Suddivisione automatica dei dati per sottogruppi con `hue`.

■ Sintassi ed Esempio Operativo

```
import seaborn as sns
sns.set_theme(style='whitegrid')
fig, ax = plt.subplots(figsize=(8, 4))
sns.barplot(data=df_clean, x='Categoria', y='Fatturato_Netto',
            hue='Canale_Vendita', errorbar=None, ax=ax)
plt.tight_layout()
```

Analisi delle Distribuzioni e Outlier: Boxplot

■ Diagnostica della Dispersione

- Anatomia Boxplot: Mediana, Box (Q1-Q3), Baffi (1.5*IQR), Outlier.
- `sns.boxplot(data=df, x='Canale', y='Sconto_Perc')` confronta le politiche sconti.
- Individua canali con concessioni di sconto fuori scala.
- Supporta il management nel controllo della marginalità.

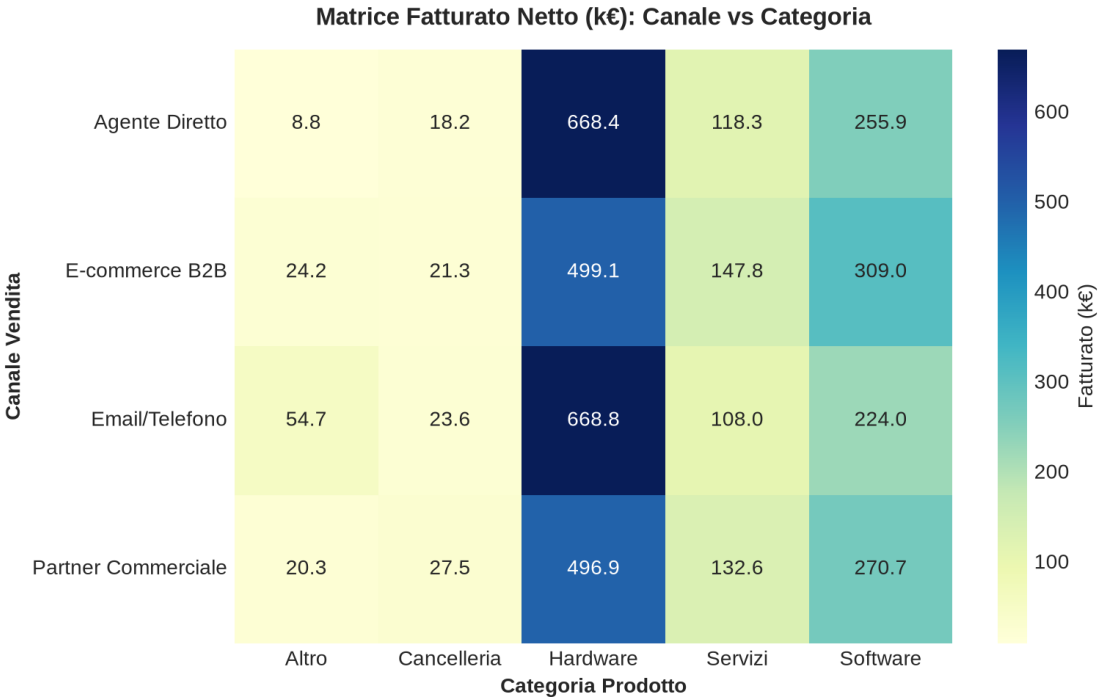
■ Sintassi ed Esempio Operativo

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=df_clean, x='Canale_Vendita', y='Sconto_Perc',
            hue='Canale_Vendita', palette='Set2', legend=False, ax=ax)
ax.set_title('Distribuzione Sconti per Canale')
```

Heatmap: Matrice Termica Canale vs Categoria

■ Mappe Termiche di Concentrazione

- Visualizza l'incrocio tra due variabili categoriche tramite colore.
- Input: Tabella Pivot con somme di fatturato in k€.
- `sns.heatmap(pivot, annot=True, fmt='.1f', cmap='YlGnBu')`.
- Individua a colpo d'occhio i cluster commerciali a maggior valore.



Composizione Multi-Grafico: Griglia 2x2 (Subplots)

■ Costruzione del Cruscotto Statico

- `fig, axes = plt.subplots(2, 2, figsize=(16, 10))` genera 4 assi indipendenti.
- `axes[0, 0]`: Trend Mensile; `axes[0, 1]`: Barre Categorie.
- `axes[1, 0]`: Boxplot Sconti; `axes[1, 1]`: Heatmap Canale x Categoria.
- Titolo generale unificato con `fig.suptitle()`.

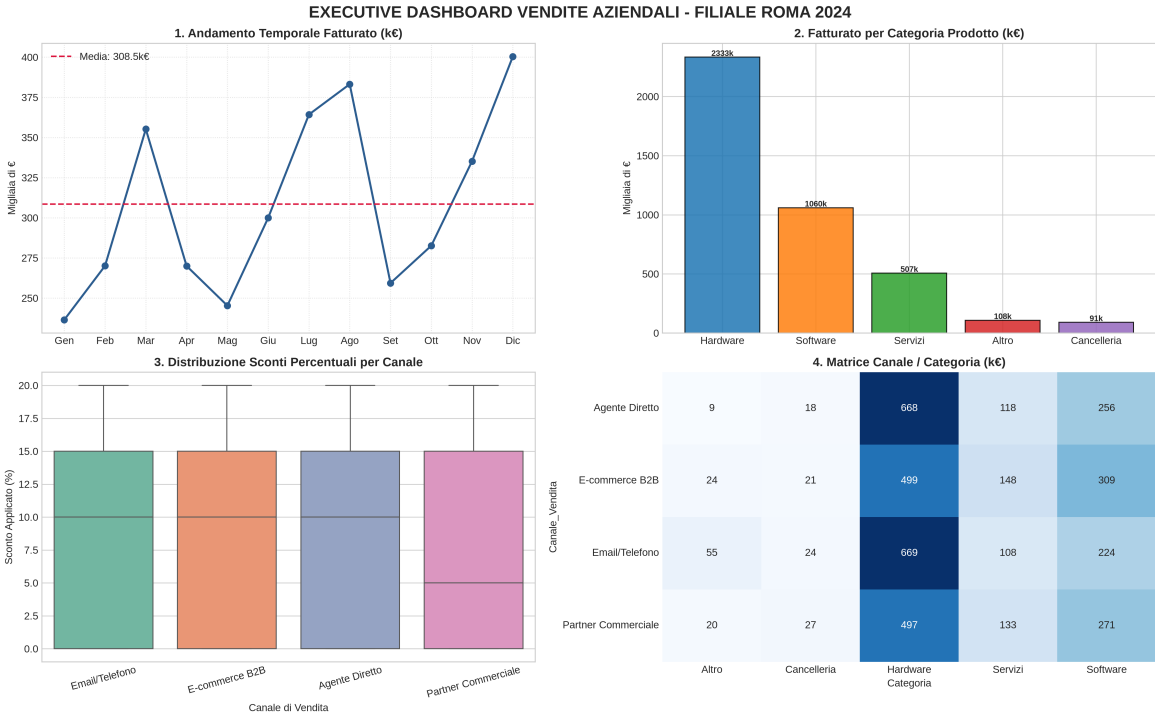
■ Sintassi ed Esempio Operativo

```
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('EXECUTIVE SALES DASHBOARD', fontsize=16, fontweight='bold')
axes[0, 0].plot(mesi, trend_val, marker='o')
axes[0, 1].bar(cat_labels, cat_val)
sns.boxplot(data=df, x='Canale', y='Sconto', ax=axes[1, 0])
sns.heatmap(pivot_matrice, ax=axes[1, 1])
plt.tight_layout()
```

Executive Dashboard 2x2 Completa per il Board

■ La Tavola Direzionale Completa

- Cruscotto direzionale integrato pronto per il CdA.
- 1. Trend temporale e stagionalità.
- 2. Composizione del catalogo vendite.
- 3. Controllo della disciplina sconti per canale.
- 4. Mappa termica della matrice distributiva.
- Esportazione a 300 DPI per stampa e reportistica PDF.



Laboratorio 4: Creazione ed Esportazione del Report Grafico

■ Esercitazione Pratica Studenti

- File: laboratori/lab04_visualizzazione/lab04_esercizi.py
- Task 1: Barplot orizzontale Top Clienti in k€.
- Task 2: Serie storica mensile con linea di media.
- Task 3: Heatmap Canale x Categoria.
- Task 4: Assemblaggio tavola 2x2 ed export ad alta risoluzione.

■ Sintassi ed Esempio Operativo

```
--- VERIFICA FILE GENERATI (dataset/generated/) ---  
■ ex4_1_top_clienti.png  
■ ex4_2_trend_mensile.png  
■ ex4_3_heatmap_matrice.png  
■ executive_report.png (300 DPI)  
plt.savefig('executive_report.png', dpi=300, bbox_inches='tight')
```

Recap Modulo 4: Visual Storytelling dei Dati Aziendali

■ Competenze Acquisite

- Architettura a oggetti Matplotlib (Figure/Axes).
- Visualizzazioni statistiche con Seaborn.
- Composizione multi-plot 2x2 per executive report.
- Esportazione ad alta definizione (300 DPI).

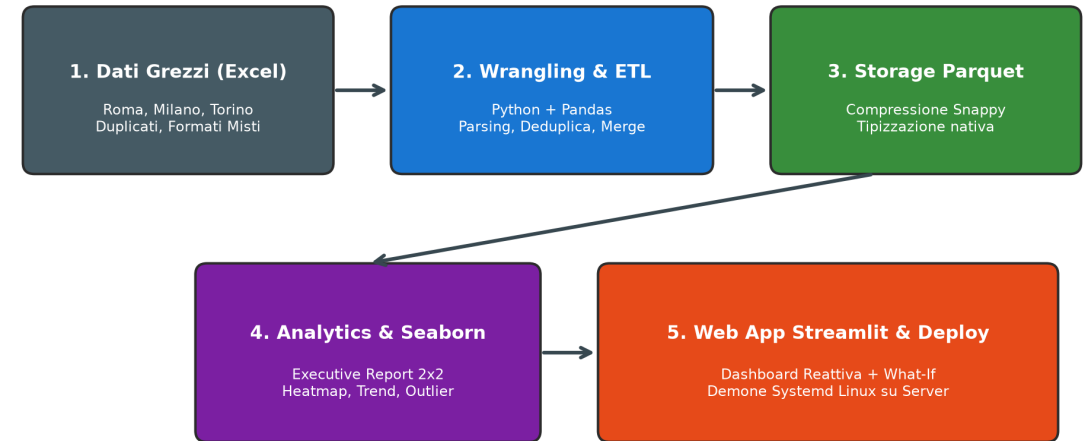
■ Sintassi ed Esempio Operativo

```
# Verso il Modulo 5: Automazione Pipeline ETL
# Automatizzeremo il flusso per processare tutte le filiali
# (Roma, Milano, Torino) con un unico script batch
# e salvataggio in formato Apache Parquet!
```

Architettura di una Pipeline ETL Aziendale

■ ■ Da Script a Processo Industriale

- Extract (E): Ingestion automatica dei file di tutte le filiali.
- Transform (T): Bonifica centralizzata, deduplica, date e join.
- Load (L): Salvataggio master Parquet e report Excel multi-scheda.
- Esecuzione in pochi secondi con un solo comando batch.



Scansione Dinamica File con glob e Percorsi Portabili

■ Gestione Dinamica del File System

- `glob.glob('dataset/raw/*.xlsx')` per trovare tutti i file regionali.
- Filtro intelligente su file temporanei lockati (~\$).
- Percorsi multiplatforma con `os.path.join()` o `pathlib`.
- Scalabilità: aggiungendo una filiale, la pipeline la include automaticamente.

■ Sintassi ed Esempio Operativo

```
import glob, os
pattern = os.path.join('dataset/raw', '*.xlsx')
files = [f for f in glob.glob(pattern) if '~$' not in f and 'report' not in f]
print(f'Trovati {len(files)} file: {[os.path.basename(f) for f in files]}')
```

Ingestion Massiva e Fusione con pd.concat()

■ Concatenazione Verticale

- Lettura ciclica dei fogli vendite e anagrafica di ciascun file.
- Gestione controllata eccezioni con try/except.
- pd.concat(lista_df, ignore_index=True) unisce i DataFrame verticalmente.
- Deduplicazione dell'anagrafica clienti master.

■ Sintassi ed Esempio Operativo

```
lista_v, lista_a = [], []  
for f in files_filiali:  
    lista_v.append(pd.read_excel(f, sheet_name='Dati_Vendite'))  
    lista_a.append(pd.read_excel(f, sheet_name='Anagrafica_Clienti'))  
df_raw_master = pd.concat(lista_v, ignore_index=True)  
df_anag_master = pd.concat(lista_a, ignore_index=True).drop_duplicates('Codice_Cliente')
```

Modularità: Centralizzazione delle Trasformazioni

■ La Funzione `trasforma_dataset()`

- Tutte le regole di bonifica incapsulate in una funzione.
- 1. Deduplicazione su transazioni e chiavi composite.
- 2. Normalizzazione testi, codici e categorie.
- 3. Parsing del 100% delle date e feature temporali.
- 4. Imputazione condizionale e merge con anagrafica.

■ Sintassi ed Esempio Operativo

```
def trasforma_dataset(df_raw, df_anag):  
    df = df_raw.drop_duplicates(subset=['ID_Transazione', 'Data_Vendita']).copy()  
    df['Categoria_Prodotto'] = df['Categoria_Prodotto'].apply(normalizza_cat)  
    df['Data_Vendita'] = pd.to_datetime(df['Data_Vendita'].apply(parse_data))  
    return df_master
```

Logging Strutturato vs Print Statement

■ Monitoraggio di Produzione

- In produzione i print() non bastano: serve tracciamento temporale e gravità.
- Modulo standard logging con formattazione oraria e livello.
- Livelli: INFO (avanzamento), WARNING (anomalie), ERROR (errori).
- Tracciamento immediato di record processati e totali generati.

■ Sintassi ed Esempio Operativo

```
import logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s] %(message)s')
logging.info('Avvio elaborazione ETL...')
logging.info(f'Fatturato Consolidato: € {tot_netto:,.2f}')
```

Storage Ottimizzato in Apache Parquet per l'Analytics

■ Il Formato Standard per i Big Data

- Apache Parquet: Formato binario colonnare compresso (Snappy).
- Compressione 10x rispetto a Excel e 5x rispetto a CSV.
- Tipizzazione nativa: i tipi datetime e float rimangono intatti.
- Velocità di lettura 10x: la dashboard carica i dati in millisecondi.

■ Sintassi ed Esempio Operativo

```
parquet_path = 'dataset/generated/vendite_consolidate_italia.parquet'  
df_master.to_parquet(parquet_path, index=False)  
df_loaded = pd.read_parquet(parquet_path) # Caricamento in 15ms!
```

Generazione Automatica del Report Excel Direzionale

■ Excel Multi-Scheda per il Management

- `pd.ExcelWriter(..., engine='openpyxl')` scrive più schede.
- Foglio 1: Dettaglio_Transazioni (Tutti i 3.600 record puliti).
- Foglio 2: Riepilogo_Filiali (KPI per sede commerciale).
- Foglio 3: Riepilogo_Categorie (Performance linee di business).
- Foglio 4: Riepilogo_Settori (Analisi clienti).

■ Sintassi ed Esempio Operativo

```
excel_path = 'dataset/generated/report_direzionale_consolidato.xlsx'
with pd.ExcelWriter(excel_path, engine='openpyxl') as writer:
    df_master.to_excel(writer, sheet_name='Dettaglio_Transazioni', index=False)
    kpi_filiali.to_excel(writer, sheet_name='Riepilogo_Filiali', index=False)
    kpi_categorie.to_excel(writer, sheet_name='Riepilogo_Categorie', index=False)
```

Recap Modulo 5: Pipeline ETL End-to-End Eseguita con Successo

■ Competenze Acquisite

- Ingestion multi-file automatica e portabile.
- Trasformazione modulare centralizzata.
- Logging strutturato di produzione.
- Doppio output: Parquet per la web app ed Excel per il CdA.

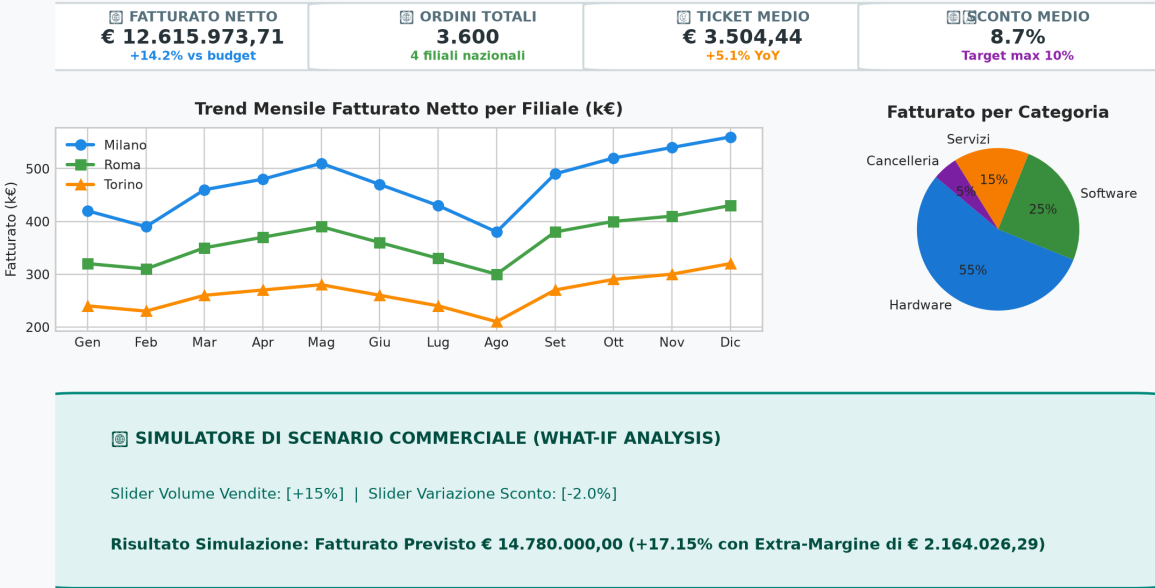
■ Sintassi ed Esempio Operativo

```
# Verso il Modulo 6: Dashboard Streamlit
# Database consolidato pronto: 3.600 record puliti
# Fatturato consolidato: € 12.615.973,71
# Costruiremo l'applicazione web interattiva con What-If!
```

Introduzione a Streamlit: Web Analytics in Pure Python

■ Il Paradigma Low-Code Python

- Streamlit trasforma script Python in web app interattive.
- Zero HTML, CSS o JavaScript richiesto.
- Modello reattivo: ad ogni interazione lo script si riesegue aggiornando la vista.
- Comando di avvio: streamlit run dashboard/app.py.



Caching ad Alte Prestazioni con @st.cache_data

■ Velocità e Ottimizzazione Memoria

- Ricaricare il Parquet dal disco ad ogni clic rallenterebbe la UI.
- @st.cache_data memorizza il dataset in RAM.
- Caricamento istantaneo in 0 millisecondi per tutti gli utenti.
- Parametro ttl=600 per invalidazione programmata ogni 10 minuti.

■ Sintassi ed Esempio Operativo

```
import streamlit as st, pandas as pd
@st.cache_data(ttl=600)
def load_dataset():
    df = pd.read_parquet('dataset/generated/vendite_consolidate_italia.parquet')
    df['Data_Vendita'] = pd.to_datetime(df['Data_Vendita'])
    return df
df_master = load_dataset()
```

Sidebar e Filtri Dinamici: st.sidebar e st.multiselect

■■ Pannello di Controllo Laterale

- st.sidebar isola i comandi di filtro dal corpo principale.
- st.multiselect() per selezionare più filiali, categorie e canali.
- Valori di default impostati a tutte le opzioni disponibili.
- Filtraggio immediato: `df[df['Filiale'].isin(sel_filiali)]`.

■ Sintassi ed Esempio Operativo

```
st.sidebar.title('■■ Filtri Commerciali')
filiali = sorted(df_master['Filiale'].unique())
sel_filiali = st.sidebar.multiselect('Filiali', filiali, default=filiali)
df_filtrato = df_master[df_master['Filiale'].isin(sel_filiali)]
```

Filtri Temporali a Calendario con st.date_input

■ Selezione Intervallo Date

- st.date_input() con range a due date (Inizio e Fine).
- Controllo di validità: applica il filtro solo quando entrambe le date sono selezionate.
- Filtraggio dinamico sulla colonna Data_Vendita.
- Contatore dei record visualizzati nella sidebar.

■ Sintassi ed Esempio Operativo

```
min_d, max_d = df_master['Data_Vendita'].min().date(), df_master['Data_Vendita'].max().date()  
sel_date = st.sidebar.date_input('Periodo Analisi', (min_d, max_d))  
if isinstance(sel_date, (list, tuple)) and len(sel_date) == 2:  
    s_d, e_d = sel_date  
    df_filtrato = df_filtrato[(df_filtrato['Data_Vendita'].dt.date >= s_d) & (df_filtrato['Data_Vendita'].dt.date <= e_d)]
```

Barra Superiore dei KPI con st.columns e st.metric

■ Executive Summary Cards

- st.columns(5) per dividere la pagina in 5 colonne affiancate.
- st.metric() mostra numeri grandi con label e delta.
- Fatturato Netto, Volume Ordini, Ticket Medio, Sconto Medio %, Pezzi Venduti.
- Formattazione monetaria con separatori di migliaia.

■ Sintassi ed Esempio Operativo

```
tot_fatt = df_filtrato['Fatturato_Netto'].sum()
tot_ord = len(df_filtrato)
c1, c2, c3, c4 = st.columns(4)
c1.metric('■ Fatturato Netto', f'€ {tot_fatt:,.2f}')
c2.metric('■ Volume Ordini', f'{tot_ord:,}')
```

Organizzazione a Schede Orizzontali con st.tabs

■ Struttura a 4 Schede Tematiche

- st.tabs() organizza contenuti complessi in modo ordinato.
- Tab 1: ■ Panoramica & Trend; Tab 2: ■ Clienti & Settori.
- Tab 3: ■ Simulatore What-If; Tab 4: ■ Dati & Export.
- Navigazione fluida e layout responsive.

■ Sintassi ed Esempio Operativo

```
tab1, tab2, tab3, tab4 = st.tabs([
    '■ Panoramica & Trend', '■ Clienti & Settori',
    '■ Simulatore What-If', '■ Dati & Export'
])
with tab1: st.subheader('Trend e Performance')
```

Tab 1: Visualizzazione Trend e Categorie con st.pyplot

■ Grafici Reattivi ai Filtri

- st.pyplot(fig) renderizza i grafici Matplotlib e Seaborn.
- Trend mensile multi-linea per filiale reattivo ai filtri.
- Barplot orizzontale del fatturato per categoria.
- Chiusura con plt.close(fig) per rilasciare memoria.

■ Sintassi ed Esempio Operativo

```
with tab1:
    c_l, c_r = st.columns([3, 2])
    with c_l:
        fig, ax = plt.subplots(figsize=(8, 4))
        # Plot curve filiali...
        st.pyplot(fig)
        plt.close(fig)
```

Tab 2: Ranking Top Clienti e Quote per Settore

■ Analisi Clientela e Mercato

- `st.dataframe()` renderizza tabelle interattive con ordinamento.
- `df.style.format({'Fatturato': '€ {:.2f}})` per valuta.
- Donut Chart delle quote di mercato settoriali.
- Ispezione dei Key Accounts e della concentrazione del business.

■ Sintassi ed Esempio Operativo

```
with tab2:
    top_c = df_filtrato.groupby('Ragione_Sociale').agg(
        Fatturato=('Fatturato_Netto', 'sum'), Ordini=('ID_Transazione', 'count')
    ).sort_values('Fatturato', ascending=False).head(10)
    st.dataframe(top_c.style.format({'Fatturato': '€ {:.2f}}'))
```

Tab 4: Esploratore Dati, Ricerca Full-Text ed Export CSV

■ Self-Service Data Discovery

- `st.text_input()` per ricerca libera multi-campo.
- Filtraggio in tempo reale durante la digitazione.
- `st.download_button()` per scaricare il dataset filtrato in CSV.
- Piena autonomia per gli utenti aziendali non tecnici.

■ Sintassi ed Esempio Operativo

```
with tab4:
    q = st.text_input('■ Cerca cliente o prodotto:')
    if q: df_filtrato = df_filtrato[df_filtrato['Ragione_Sociale'].str.contains(q, case=False, na=False)]
    st.dataframe(df_filtrato.head(100))
    csv = df_filtrato.to_csv(index=False).encode('utf-8')
    st.download_button('■ Scarica CSV', csv, 'export.csv', 'text/csv')
```

Tab 3: Simulatore Decisionale di Scenario ('What-If Analysis')

■ Strumento di Supporto alle Decisioni

- Simulatore di scenario commerciale per il management.
- Slider 1: Variazione volume vendite (+/- 50%).
- Slider 2: Variazione punti percentuali di sconto (+/- 15%).
- Ricalcolo istantaneo del fatturato previsto con delta monetario e %.

■ Sintassi ed Esempio Operativo

```
with tab3:
    d_vol = st.slider('■ Variazione Volume (%)', -50, 50, 0, 5)
    d_sco = st.slider('■ Variazione Sconto (pt %)', -15, 15, 0, 1)
    qta_s = df_filtrato['Quantita'] * (1 + d_vol/100.0)
    sco_s = (df_filtrato['Sconto_Perc'] + d_sco).clip(0, 100)
    fatt_s = (qta_s * df_filtrato['Prezzo_Unitario'] * (1 - sco_s/100)).sum()
    st.metric('Fatturato Previsto', f'€ {fatt_s:,.2f}', delta=f'{fatt_s - tot_fatt:+, .2f} €'
    )
```

Personalizzazione Grafica e Custom CSS in Streamlit

■ Look & Feel Aziendale Professionale

- Iniezione CSS personalizzato con `st.markdown(..., unsafe_allow_html=True)`.
- Card shadow, bordi colorati e allineamento alla brand identity.
- Loghi aziendali nella sidebar con `st.sidebar.image()`.
- Badge e banner informativi con `st.info()`, `st.success()`, `st.warning()`.

■ Sintassi ed Esempio Operativo

```
st.markdown('''  
<style>  
  .main-metric-box { background-color: #f8f9fa; border-left: 5px solid #1E88E5; padding: 1  
2px; }  
</style>  
''', unsafe_allow_html=True)
```

Laboratorio 6: Sviluppo della Web App Aziendale Completa

■ Esercitazione Pratica Studenti

- File: laboratori/lab06_dashboard_streamlit/app_starter.py
- Task 1: Caching Parquet con @st.cache_data.
- Task 2: Sidebar con multiselect e date_input.
- Task 3: 5 KPI Cards superiori.
- Task 4: Sviluppo 4 tabs (Trend, Clienti, What-If, Dati & Export CSV).

■ Sintassi ed Esempio Operativo

- ```
--- VERIFICA FUNZIONALITÀ DASHBOARD ---
```
- Web Server attivo su `http://localhost:8501`
  - Caching Parquet attivo (`< 20ms`)
  - Filtri sidebar interattivi operativi
  - 5 KPI Cards dinamiche allineate
  - Simulatore What-If real-time operativo
  - Ricerca e download CSV funzionanti

## Recap Modulo 6: Dalla Tabella Statica alla Web Intelligence

### ■ Competenze Acquisite

- Applicazione web interattiva in puro Python.
- Caching dei dati in RAM per massime prestazioni.
- Dashboard decisionale con simulatore What-If.
- Ricerca full-text e self-service download dati.

### ■ Sintassi ed Esempio Operativo

```
Verso il Modulo 7: Deploy Linux 24/7
Trasformeremo l'applicazione in un servizio demone
continuo su server Linux con gestione Systemd,
avvio automatico al boot e monitoraggio log!
```

# Architettura di Produzione su Server Linux 24/7

## ■ Dal Laptop Locale al Server Aziendale

- Obiettivo: Dashboard sempre attiva 24/7 per tutti i colleghi.
- Server Linux Ubuntu con Virtual Environment dedicato.
- Architettura a 3 livelli: Browser → Nginx Reverse Proxy → Demone Systemd.
- Sessione in modalità FULL LIVE DEMO & Show-and-Tell.



⚙ Gestione con `systemctl` (start/stop/status) e log con `journalctl -f`

# Configurazione del Demone Systemd: dashboard\_vendite.service

## ■ Anatomia del File Unit di Sistema

- File: /etc/systemd/system/dashboard\_vendite.service.
- Sezione [Service]: Utente non-root, WorkingDirectory, ExecStart.
- Resilienza: Restart=always e RestartSec=5 (auto-riavvio su crash).
- Sezione [Install]: WantedBy=multi-user.target per boot automatico.

## ■ Sintassi ed Esempio Operativo

```
/etc/systemd/system/dashboard_vendite.service
[Unit]
Description=Dashboard Vendite ITIS Campobasso
After=network.target
[Service]
Type=simple | User=arny
ExecStart=/path/.venv/bin/streamlit run dashboard/app.py --server.port 8501
Restart=always | RestartSec=5
[Install] | WantedBy=multi-user.target
```

# Amministrazione del Servizio con systemctl e Script di Provisioning

## ■ Comandi Operativi di Amministrazione

- `sudo systemctl daemon-reload`: Ricarica configurazioni.
- `sudo systemctl enable dashboard_vendite`: Abilita avvio al boot.
- `sudo systemctl start / restart / stop`: Gestione ciclo di vita.
- `sudo systemctl status`: Ispezione stato attivo (active / running).

## ■ Sintassi ed Esempio Operativo

```
$ sudo systemctl status dashboard_vendite.service
● dashboard_vendite.service - Dashboard Vendite
 Active: active (running) since Mon 10:00:00 CEST
 Main PID: 42150 (streamlit)
 Memory: 142.5M | CPU: 1.250s
```

# Monitoraggio dei Log con journalctl & Cenni Nginx

## ■ Diagnostica e Sicurezza Perimetrale

- journalctl -u dashboard\_vendite.service -f: Streaming log in tempo reale.
- Ispezione errori e diagnostica connessioni utenti.
- Reverse Proxy Nginx su porta standard 80/443 con certificato SSL.
- Guida operativa completa in [laboratori/lab07\\_deploy\\_linux/guida\\_deploy.md](#).

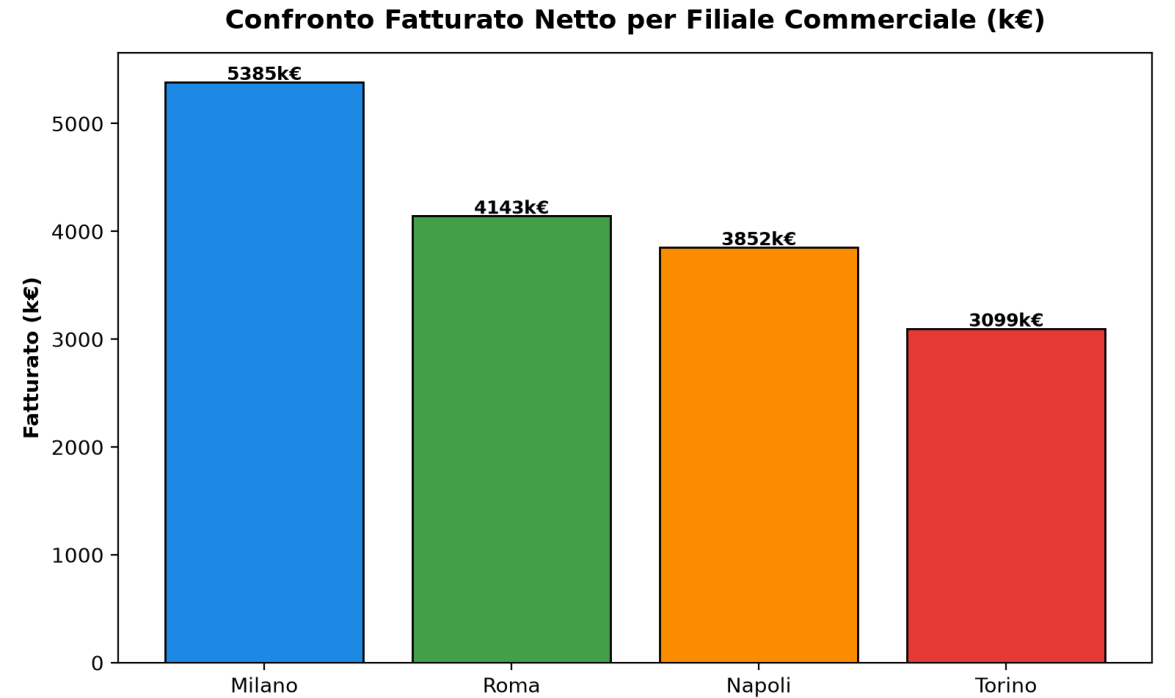
## ■ Sintassi ed Esempio Operativo

```
$ sudo journalctl -u dashboard_vendite.service -f -n 20
10:00:01 [INFO] Uvicorn server started on 0.0.0.0:8501
10:00:05 [INFO] Caching data: 3600 records loaded
10:01:12 [INFO] User session started: IP 192.168.1.56
```

## Project Work Finale: Acquisizione Filiale di Napoli (3H)

### ■ La Sfida Aziendale Conclusiva

- Scenario: Apertura della nuova sede commerciale di Napoli.
- Dataset: dataset/raw/napoli\_project\_work.xlsx (1.144 record grezzi).
- Missione: Consolidamento a 4 filiali nella pipeline nazionale.
- Risposta ai quesiti strategici e report per la Direzione Generale.



# Specifiche Tecniche del Project Work & Rubrica di Valutazione

## ■ Le 4 Fasi del Progetto

- Fase 1 (25 pt): Data Wrangling su Napoli.
- Fase 2 (25 pt): Consolidamento nazionale a 4 filiali (4.700 record totali).
- Fase 3 (25 pt): Calcolo KPI di business e benchmark.
- Fase 4 (25 pt): Produzione dei 3 grafici di benchmark e aggiornamento dashboard.

## ■ Sintassi ed Esempio Operativo

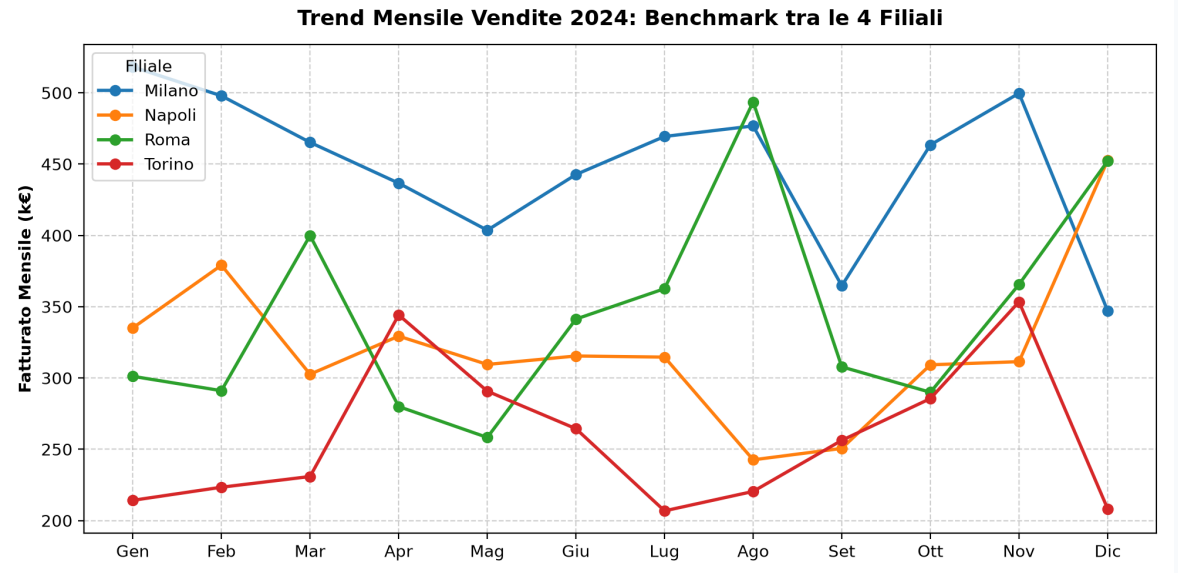
--- GRIGLIA DI VALUTAZIONE (100 PUNTI) ---

|                                   |           |
|-----------------------------------|-----------|
| • Data Wrangling & Pulizia Dati:  | 25 Punti  |
| • Integrazione & Consolidamento:  | 20 Punti  |
| • Accuratezza Calcoli e KPI:      | 20 Punti  |
| • Visualizzazione e Storytelling: | 20 Punti  |
| • Qualità Codice PEP 8:           | 15 Punti  |
| TOTALE:                           | 100 Punti |

## Sessione di Sviluppo Guidato e Supporto in Aula

### ■ Lavoro Pratico Autonomo (2 Ore)

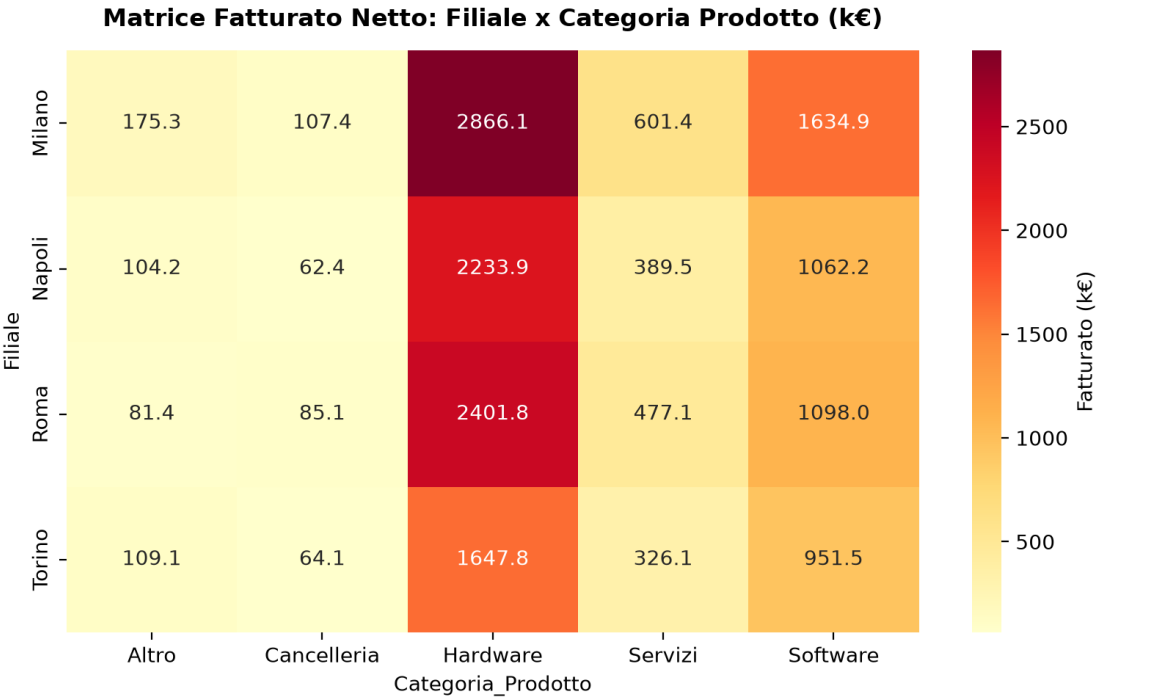
- Gli studenti sviluppano attivamente la pipeline e i grafici.
- Supporto personalizzato del docente e troubleshooting.
- Totale record master a 4 filiali: esattamente 4.700 righe pulite.
- Fatturato nazionale consolidato: oltre 16.4 Milioni di Euro.



# Presentazione Risultati: Benchmark Nazionale a 4 Filiali

## Risposte ai Quesiti di Business

- 1. Quote Nazionali:
  - Milano: € 5.385k (32.7%) | Roma: € 4.143k (25.1%)
  - Napoli: € 3.852k (23.4%) | Torino: € 3.098k (18.8%)
- 2. Performance Napoli:
  - Categoria Top: Hardware (€ 2.233k) con 3.012 pezzi.
  - Canale Top: Partner Commerciale (€ 1.118k).
- 3. Napoli si attesta come 3° polo commerciale nazionale.



# Soluzione Docente, Certificazione & Chiusura Corso

## ■ Competenze Certificate Raggiunte

- 1. Python Operativo & Funzioni Commerciali.
- 2. Pandas Mastery & Vettorializzazione.
- 3. Data Wrangling & JOIN Relazionali.
- 4. Visual Storytelling (Matplotlib/Seaborn 300 DPI).
- 5. Pipeline ETL con Apache Parquet.
- 6. Web App Streamlit con What-If.
- 7. Deploy Linux Systemd 24/7.

## ■ Sintassi ed Esempio Operativo

### ■ Materiali disponibili su GitHub:

[https://github.com/arnymore/python\\_analisi\\_dati\\_campobasso](https://github.com/arnymore/python_analisi_dati_campobasso)

Soluzione: `project_work/soluzione_project_work.py`

Manuale: `dispensa/indice_dispensa.md`

### ■ Congratulazioni a tutti i partecipanti!